
Chapter 19 Functional Programming with Java 8

19.1 Effect-Free Programming

19.2 First-Class Functions

- [Lambda Expressions !\[\]\(38441ceaa711016e0bf2ad46ad394ff4_img.jpg\)](#)

19.3 Streams

- [Basic Idea !\[\]\(30a147af384f9f71632c2ff17bc706c8_img.jpg\)](#)
- [Using Map !\[\]\(9b33568d5c136f08ca688ce48be37574_img.jpg\)](#)
- [Using Filter !\[\]\(8c93063dab026f10e159986b27c41c64_img.jpg\)](#)
- [Using Reduce !\[\]\(8a17676a8da87a4e59299223a765e613_img.jpg\)](#)
- [Optional Results !\[\]\(f7fdc7cc047b770fc5fdd2c2137c07d9_img.jpg\)](#)

19.4 Function Closures

19.5 Higher-Order Operations on Collections

- [Working with Arrays !\[\]\(49aa2e1da5fe39294864e9598c593810_img.jpg\)](#)
- [Working with Lists !\[\]\(7d0a8d8b1031f74abe67b09fcf4a2322_img.jpg\)](#)

- [Working with Files](#) 

19.6 Case Study: Perfect Numbers

- [Computing Sums](#) 
- [Incorporating Square Roots](#) 
- [Just Five and Leveraging Concurrency](#) 

Introduction

Every 10 to 15 years the computer science education community updates the suggested curriculum for computer science majors. In the most recent curriculum revision in 2013, the ACM/IEEE joint task force updated the suggestions for coverage of programming languages. As they explained in their introduction to that section:

Software developers must understand the programming models underlying different languages and make informed design choices in languages supporting multiple complementary approaches. Computer scientists will often need to learn new languages and programming constructs, and must understand the principles underlying how programming language features are defined, composed, and implemented.

*They went on to describe specific concepts from various programming paradigms that should be studied by every undergraduate computer science major. Two of the paradigms that they emphasize are the procedural and object-oriented paradigms that have been discussed thoroughly in this book. Java is a hybrid language that supports both approaches. We began the book by studying procedural programming with static methods that we used to decompose a large program into action-oriented components. Then starting in **Chapter 8** we moved on to the object-oriented approach in which a large program is decomposed into a set of interacting objects each with their own class defining their state and behavior.*

In this chapter, we will explore a third approach to programming that is known as *functional programming*. Java uses the term “method” for what many other languages call a “function,” so in the discussion that follows, you can mentally substitute “method” for “function.”

Functional Programming

A style of programming that emphasizes the use of functions (methods) to decompose a complex task into subtasks.

The Java community has spent years discussing how best to add functional features to the language, culminating with the release of Java 8 in 2014. The new features added to the language in this release allow Java programmers to take advantage of this different approach to programming. The new curriculum guidelines list five specific topics that undergraduates should study related to functional programming:

1. Effect-free programming
2. Processing structured data via functions
3. First-class functions
4. Function closures
5. Higher-order operations on collections

The first three are categorized as Core Tier-1 and items 4 and 5 are categorized as Core Tier-2. As the guidelines explain, “computer-science curricula should cover all the Core Tier-1 topics” and “all or almost all of the Core Tier-2 topics.” Each of these concepts has its own section in this chapter, except for Item 2 which does not translate easily to Java. In addition, this chapter includes a separate section that discusses the central Java 8 concept of streams because much of the functional programming support is provided using that mechanism. This chapter ends with a short case study that picks up an additional Core Tier-1 concept of using concurrency to speed the execution of a complex computation. As we will see, the functional programming approach is particularly well suited to safe and reliable concurrency, which is becoming increasingly important in modern computing where the average PC now comes with multiple processors (sometimes described as “multi-core processing”). The terminology can sometimes be intimidating, but as we will see, these concepts are fairly straightforward to discuss using Java 8 constructs.

19.1 Effect-Free Programming

Consider the following line of code:

```
int result = f(x) + f(x);
```

This code computes a result by calling a function (a method) two different times and adding together the results. Offhand it seems that we would get the exact same behavior with this line of code:

```
int result = 2 * f(x);
```

Sometimes the two are equivalent and sometimes not. The question to consider is whether the function produces a *side effect*.

Side Effect

A change to the state of an object or program variable produced by a call on a function (i.e., a method).

As an example, consider the following program:

```
1  // This program demonstrates a method with a side effect.
2  // Each time f is called, the variable x's value changes.
3
4  public class SideEffect {
5      public static int x;
6
7      public static int f(int n) {
8          x = x * 2;
9          return x + n;
10     }
11
12     public static void main(String[] args) {
13         x = 5;
14         int result = f(x) + f(x);
15         System.out.println(result);
16     }
17 }
```

This confusing program begins with the declaration of a static variable called `x` that has the entire class as its scope. Such variables are called *global variables*. We have discouraged the use of global variables because they can lead to the kind of confusing code we have here.

The program begins by initializing the global variable `x` to the value 5. Then on line 14 the program calls the function `f` twice, passing it the value of `x` each time. But `f` doubles the value of `x` before returning the sum of the value passed as a parameter and `x`. If you find this all highly confusing, you're not alone. This style of programming is very bad.

On the first call to `f`, the variable `x` has the value 5. This is passed from `main` as the parameter called `n`. Inside function `f`, we first double `x`, which gives it the value 10. It then returns the sum of `x`, which is now 10, and `n`, which is 5. So it returns 15 for the first call. On the second call, it passes the current value of `x`, which is 10, to the function and uses this to initialize `n` to be 10. Then it doubles `x` from 10 to 20 and returns the sum of `x` and `n`, which are 20 and 10. So it returns the value 30 on the second call. The variable `result` is set to the sum of these two returns values, which is 15 plus 30, or 45.

What happens if we substitute our other line of code for setting `result`?

```
int result = 2 * f(x);
```

The first call on the function behaves as it did before, returning the value 15. We double that to 30 and set `result` to 30. So we end up

with a program that has different behavior because the function `f` has a side effect.

Because functional programming focuses so much on individual functions, the community of programmers who use functional programming regularly have concluded that side effects should be avoided when possible, leading to the kind of “effect free” functions that serve as the title of this section.

The object-oriented community has been less wary of functions with side effects. A central idea of OOP is that objects have state that changes as methods are called. Recall from [Chapter 8](#) that instance methods are classified as *accessors* that can be thought of as read-only operations accessing the object’s state and *mutators* that can be thought of as read/write operations that often change the state of the object. Using the terminology from functional programming, we would say that accessors are usually effect-free functions while mutators are functions with a side effect. Even accessors can have side effects because they might change the state of other objects.

One of the simplest and most pervasive sources of side effects we have seen is the printing of values. Continuing our example of a function `f` that might be called once or twice, if the function includes a call on `System.out.println`, then it will produce more output when called twice than when it is called once. So even though the variable `result` in our example might be set to the same value, there is still a possible side effect caused by the function if it produces output.

As we will see in the case study at the end of this chapter, there are great advantages to writing code that is free of side effects. It is easier to take advantage of concurrency using such code and it is easier to prove formal properties of programs that are written using the effect-free approach.

DID YOU KNOW?

Functional Programming Influences on Java

One of the most successful functional programming languages is the Scheme language developed at MIT in the 1970s by Guy Steele and Gerry Sussman. Jim Gosling who designed the Java programming language was well aware of Scheme and it is clear that many of the ideas from Scheme influenced the design of Java.

As mentioned in [Chapter 3](#), objects of the `String` class are immutable, which means that all of their methods are free of side effects. This has greatly simplified the use of `String` objects. Consider, for example, a class with a field called `name` of type `String` and an accessor method for examining the value of that field:

```
public String getName() {  
    return name;  
}
```

In Java, you don't have to worry about the possibility that a client of this class might alter your string object. In other programming languages, you might make a defensive copy to give to the client to avoid any possible interference.

The idea of effect-free programming is also evident in the `Collections` class that has methods for constructing an “unmodifiable” version of a collection, such as

```
Collections.unmodifiableList.
```

Another example of the influence of Scheme can be found in Java's concept of interning of strings (read the description of the method `String.intern` if you are curious). It is also worth noting that Guy Steele, the co-designer of Scheme, was hired by Sun Microsystems in 1994 to work on the Java team.

19.2 First-Class Functions

When you study any particular programming language, you will soon learn what the basic elements of the language are. For example, almost every programming language allows you to manipulate numbers as basic elements. You can store a number in a variable, you can pass a number as a parameter, you have various operations for manipulating numbers, and you can return a number from a method call. Strings are also basic elements of Java because they are easy to declare, store, and pass as parameters.

Not every data type is a first-class citizen of every language. For example, not every programming language allows you to manipulate Boolean values as basic elements the way Java does. For decades the C programming language did not have a proper type for storing Boolean values and programmers instead used the integer values 1 and 0 to represent true and false. Some languages consider collections like lists and maps to be first-class citizens, providing special syntax for declaring and manipulating them; in Java, arrays receive first-class treatment and special syntax, but the collection objects do not.

First-Class Citizen

An element of a programming language that is tightly integrated with the language and supports the full range of operations generally available to other entities in the language.

What would it mean to provide first-class status for functions? Let's begin with an example that points out the motivation. Consider the task of performing drill and practice with a user where you make up addition problems involving two numbers between 1 and 12 and you keep track of how many answers the user got right. Below is a typical interaction.

```
10 + 6 = 16
you got it right
9 + 6 = 15
you got it right
3 + 7 = 9
incorrect...the answer was 10
12 + 10 = 22
you got it right
9 + 12 = 20
incorrect...the answer was 21
3 of 5 correct
```

Suppose that the above quiz is represented as a method called `giveProblems` that is passed a `Scanner` that reads from the console and a number of problems to provide to the user, as in:

```
Scanner console = new Scanner(System.in);  
giveProblems(console, 5);
```

This is a straightforward task to solve building on material from **Chapters 1** through **5**.

```
public static void giveProblems(Scanner console, int  
numProblems) {  
    Random r = new Random();  
    int numRight = 0;  
    for (int i = 1; i <= numProblems; i++) {  
        int x = r.nextInt(12) + 1;  
        int y = r.nextInt(12) + 1;  
        System.out.print(x + " + " + y + " = ");  
        int answer = x + y;  
        int response = console.nextInt();  
        if (response == answer) {  
            System.out.println("you got it right");  
            numRight++;  
        } else {  
            System.out.println("incorrect...the answer was " +  
answer);
```

```
    }  
    }  
    System.out.println(numRight + " of " + numProblems + "  
correct");  
}
```

We now have a method that allows us to have a user practice addition problems. What if we wanted to also practice multiplication problems? Most of the code stays the same, but we would have to change the addition in these two lines of code:

```
System.out.print(x + " + " + y + " = ");  
int answer = x + y;
```

to be multiplication:

```
System.out.print(x + " * " + y + " = ");  
int answer = x * y;
```

In a sense, we want to replace the plus with an asterisk in both of these lines of code. If we want the code to work for both addition and multiplication, the first line of code is easier to generalize. The only part that needs to change is the text to be printed, and we have the ability to manipulate text as a basic element of Java using a `String`.

We can change the method header to take a third parameter specifying the text to print:

```
public static void giveProblems(Scanner console, int
numProblems,
    String text) {
    ...
    System.out.print(x + " " + text + " " + y + " = ");
    ...
}
```

We can then make two calls on the method to have it use a plus the first time and an asterisk the second time:

```
giveProblems(console, 5, "+");
giveProblems(console, 5, "*");
```

So far so good. But now we run into the problem that the method is computing the right answer as the sum of the two numbers, even if an asterisk is passed as the text. That will make for a very frustrating experience on the second call where the console indicates to the user that the problem is a multiplication problem.

The question is how to change the computation of the right answer. It would be great if we could say:

```
int answer = x text y;
```

If Java somehow filled in “+” or “*” appropriately and then used the corresponding operator for the computation, then it would work. But Java doesn’t work this way. The way that we usually get around this in Java is by introducing an `if/else` structure that tests whether the string is a plus or an asterisk. But then the code works for only those two operators, and additional branches must be added to the code later to make it support subtraction and other operations.

What we really want is the ability to pass an additional parameter that specifies the calculation to perform. We want to say, “Use the addition operation the first time and the multiplication operation the second time.” A functional programmer would say that what we want to be able to do is to pass in a function. This is an example of what we mean by elevating functions to first class status in the language. We want to be able to introduce a fourth parameter that specifies the function to use for computing the right answer. That requires thinking of the function as a thing in the language that can be passed as a parameter.

Lambda Expressions

Java 8 provides a nice mechanism for doing exactly that. We can form a *lambda expression*.

Lambda Expression (Lambda)

An expression that describes a function by specifying its parameters and the value that it returns.

The term “lambda” was coined by a logician named Alonzo Church in the 1930s. The term is used consistently across many programming languages, so it is worth becoming familiar with it. The Python programming language, for example, uses “lambda” as a keyword for forming this type of anonymous function.

Lambda expressions are formed in Java by specifying the parameters of the function and an expression that represents the value to return separated by the special operator “->”.

```
<parameters> -> <expression>
```

For example, we can use the following lambda expression to represent a function that adds together two arguments:

```
(int x, int y) -> x + y
```

Notice that the parameters are enclosed in parentheses. In reading this expression, we typically describe it as, “Given the parameters x and y of type `int`, we return $x + y$.” We can also write this as a method with a name, as in:

```
public static int sum(int x, int y) {  
    return x + y;  
}
```

Notice how the lambda expression takes the parenthesized parameter list from the method header and the expression used in the return statement to form a simple expression. Once you get used to reading lambda expressions, you will find that it is a concise way to read and reason about the underlying computation being performed.

It is also often possible to eliminate the types for the parameters because they can usually be inferred by the surrounding context. For our sample code, we will be able to use this lambda expression to describe addition:

```
(x, y) -> x + y
```

And this expression to describe multiplication:

```
(x, y) -> x * y
```

Given this new option, we can rewrite our client code as follows to perform 3 each of addition and multiplication problems.

```
Scanner console = new Scanner(System.in);  
giveProblems(console, 3, "+", (x, y) -> x + y);  
giveProblems(console, 3, "*", (x, y) -> x * y);
```

Below is a sample log of execution.

```
9 + 1 = 10  
you got it right  
4 + 4 = 8  
you got it right  
6 + 2 = 9  
incorrect...the answer was 8  
2 of 3 correct  
  
10 * 11 = 110  
you got it right  
9 * 6 = 64  
incorrect...the answer was 54  
5 * 7 = 45
```

```
incorrect...the answer was 35  
1 of 3 correct
```

This ability to pass a lambda expression as a parameter points out the benefit of treating functions as first-class elements of the language. Just as we can provide a different number of problems to perform or a different text to use for displaying the problems, we can also provide a different function for computing the right answer. This is a much more flexible approach than having to write tedious `if/else` constructs that say exactly what to do for each different possibility. Instead we provide a simple definition of the function we want to use and the function is stored in a parameter of the method.

It is more important that you learn how to become a client of these functional programming features of Java than to learn how to implement them yourself. But for those who are interested in seeing the implementation, the following is the revised method code:

```
public static void giveProblems(Scanner console, int  
numProblems,  
                                String text, IntBinaryOperator  
operator) {  
    Random r = new Random();  
    int numRight = 0;  
    for (int i = 1; i <= numProblems; i++) {  
        int x = r.nextInt(12) + 1;  
        int y = r.nextInt(12) + 1;
```

```

        System.out.print(x + " " + text + " " + y + " = ");
        int answer = operator.applyAsInt(x, y);
        int response = console.nextInt();
        if (response == answer) {
            System.out.println("you got it right");
            numRight++;
        } else {
            System.out.println("incorrect...the answer was " +
answer);
        }
    }

    System.out.println(numRight + " of " + numProblems + "
correct");
    System.out.println();
}

```

There are other variations on lambda expression syntax. For example, if a lambda expression accepts only a single parameter, the parentheses around it are not required. The following is a lambda expression that accepts an integer and returns that integer plus 1:

```
n -> n + 1
```

Another syntax variation is that if the computation is not a simple expression, you can include multiple statements enclosed in curly braces, such as:

```
x -> { int z = x * x; System.out.println(z); return z; }
```

Our discussion of first-class functions is a little generous to Java because it turns out that functions in Java 8 are not truly first-class. The language designers have done some fancy work behind the scenes to make it feel like Java has first-class functions, but they aren't really first-class because you can't do basic things like storing them directly in a variable. Instead Java takes advantage of interfaces that have a single abstract method in them (known as *functional interfaces*) and constructs an object for you that implements the interface's method using the elements of the lambda expression. As a result, Java's implementation of functional programming is more clunky and restrictive than in a true functional programming language. But for our purposes, the lambda expressions and functional interfaces act enough like first-class functions that we can explore the concept, even though it is a bit of an illusion.

You can try out lambdas in JShell, but the syntax is a bit clunky compared to some other programming languages. The following JShell interaction creates a lambda function called `f` and calls it on two integers:

```
jshell> IntBinaryOperator f = (x, y) -> x + y;  
f ==> $Lambda$14/1551870003@39aeed2f
```

```
jshell> f.applyAsInt(6, 2)
```

```
$2 ==> 8
```

19.3 Streams

You don't truly appreciate the benefits of functional programming in Java until you explore streams, which are the primary mechanism that Java provides for this style of programming. We have seen this concept before when we studied files in [Chapter 6](#) . We saw input streams that are a source of data and output streams that are a destination for results. The streams in this chapter are a generalization of that idea. Oracle describes a Java stream as a sequence of elements of data on which various functional programming operations can be performed.

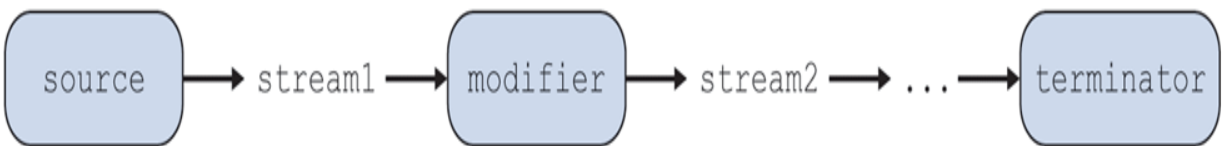
Stream

A sequence of elements from a data source that supports aggregate operations.

Basic Idea

The best way to think about a stream is to visualize it as a flow of data from a source to a terminator with possible modifiers in between, as shown in [Figure 19.1](#) .

Figure 19.1 Streams



There is always one source and one terminator, but there can be any number of modifiers (including none) in between. As the diagram indicates, think of each modifier as transforming the stream in some way. One sequence of values flows in and a different sequence of values flows out. This way, we solve a complex programming task by identifying the source of the data to process, the final result we want to compute, and a series of transformations in between that move us closer to completing the task. Each of the modifiers will be specified by a function, which means that we are decomposing the overall task into a series of subtasks that each involve a single transformation specified by a function.

As a first example, suppose we want to find the sum of the squares of the integers 1 through 5. We could use a classic cumulative sum to accomplish this:

```
// compute the sum of the squares of integers 1-5
int sum = 0;
```

```
for (int i = 1; i <= 5; i++) {  
    sum = sum + i * i;  
}
```

This code specifies exactly how to perform this computation, using a loop variable called `i` that varies from 1 to 5 and accumulating the final answer in a variable called `sum`. You will see that when we use streams, we describe more *what* we want computed instead of specifying *how* to compute it. This can make the coding itself simpler, but more importantly, it gives the computer more flexibility to decide how to implement the computation. As we will see in the case study at the end of this chapter, this can allow the computer to optimize the solution to run faster.

Using a stream approach, we first have to identify a source of data. We don't have a convenient source for the squares of the positive integers, but there is a static method called `IntStream.range` that produces a stream of sequential integers in a particular range. As with the `substring` method of the `String` class, the `range` method has a first parameter that is inclusive and a second parameter that is exclusive. So we will make the call `IntStream.range(1, 6)` to produce a stream with the integers [1, 2, 3, 4, 5]. There is a variation of the method called `rangeClosed` that would allow us to pass `(1, 5)` as parameters, but the `range` method uses the same convention we have studied in Java for substrings and it is also more commonly used in other programming languages. Python, for example, has a `range` function that works the same way.

We also need to pick an appropriate terminator. In this case Java provides one for us in the form of a method called `sum` that adds up the values in a stream of numbers. For now, let's just add up the integers and store the result in a variable. So we would write this line of code:

```
int sum = IntStream.range(1, 6).sum();
```

This sets the variable `sum` to 15 ($1 + 2 + 3 + 4 + 5$). In this case, we have the required source of data and the required terminator, but there are no modifications along the way. [Figure 19.2](#)  shows a diagram of what is going on.

Figure 19.2 Stream operations on range of integers

`IntStream.range(1, 6) -> [1, 2, 3, 4, 5] -> sum -> 15`

In our computation, the initial call to `IntStream.range` is the source, and `sum` is the terminator. The call on `range` produces a stream of five integers. This stream is fed into the `sum` method, which adds them up to produce the final result of 15. But recall that we want the sum of the first five squares, not the first five integers. We can accomplish that by introducing a modifier in between the range creation and the sum operation.

You can see optional values in JShell. It displays that the result is optional, along with its value, if any. If you want to split up long lines

with multiple stream operations, you have to type the dot character before you press Enter for JShell to know you haven't finished writing the stream expression yet:

```
jshell> IntStream.of(55, 20, 19, 31, 40, -2, 62, 30).  
...> filter(n -> n % 10 == 0).  
...> max()  
$1 ==> OptionalInt[40]  
  
jshell> $1.getAsInt()  
40
```

Using Map

Stream objects have a method called `map` that takes a function as a parameter. We can provide a function that squares a number. What `map` does is to produce a new stream that has the result of applying the given function to each element of the original stream. So if the original stream had five numbers, then the new stream will also have five numbers, but they will be new numbers obtained by calling the function passed as a parameter on each of the five elements in the stream. So our line of code becomes:

```
int sum = IntStream.range(1, 6).map(n -> n * n).sum();
```

There is a convention popular among Java programmers to list each step of this operation on a different line of code. The first line of code should have the source and the final line should have the terminator and any modifications should be listed as separate lines in between. So the code above becomes:

```
int sum = IntStream.range(1, 6)
    .map(n -> n * n)
    .sum();
```

This is just a formatting convention to make it easier to read the code. We would read this in a high-level way as, “To assign the variable `sum`, first form the given range of integers, then map the given function over those integers, and then find their sum.” We can update our diagram as shown in [Figure 19.3](#), again using the convention of including one step on each line.

Figure 19.3 Stream operations with `map`

```
IntStream.range -> [1, 2, 3, 4, 5]
            -> map -> [1, 4, 9, 16, 25]
            -> sum -> 55
```

As the diagram indicates, it is useful to think in terms of a flow of data. First the call on `range` produces the sequence [1, 2, 3, 4, 5].

This is then passed through the `map` modifier that changes the stream by applying the squaring function to each value, generating the new stream [1, 4, 9, 16, 25]. This new stream is then sent into the `sum` terminator that adds them up to produce the value 55.

This example has the classic structure of one source of data, one modification, and one terminator that computes a result. It is useful to keep in mind these three basic elements of a typical stream computation. But you can have more than one modification along the way, so let's look at an example that has many.

Using Filter

Suppose that instead of using sequential integers, we decide to work with the first 10 digits of π . We can form a specific stream of integers by calling the method `IntStream.of` and listing the individual values, so let's use the same code as before but using those 10 digits:

```
int sum = IntStream.of(3, 1, 4, 1, 5, 9, 2, 6, 5, 3)
    .map(n -> n * n)
    .sum();
```

This sets `sum` to be 207, as indicated in [Figure 19.4](#) .

Figure 19.4 Stream operations on digits of π

```
IntStream.of -> [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
            -> map -> [9, 1, 16, 1, 25, 81, 4, 36, 25, 9]
            -> sum -> 207
```

But suppose that we don't want to add up the squares of all of these digits. Suppose we are only interested in the odd digits. We could rewrite the function we used in `map` to return 0 for even numbers and the square of the number for odd numbers, but there is a better way. There is a modifier known as `filter` that can be used to restrict a stream to those values that pass some test. So while the `map` modifier gives you a stream of new values of the same length as the original, the `filter` modifier gives you a stream of unchanged values, but not necessarily of the same length because not all values pass the given test.

The `filter` method takes as its argument a Boolean-valued predicate. In our case, we will test that the remainder when divided by 2 is not 0.

```
int sum = IntStream.of(3, 1, 4, 1, 5, 9, 2, 6, 5, 3)
    .filter(n -> n % 2 != 0)
    .map(n -> n * n)
    .sum();
```

Updating our diagram for the flow of data we get the result shown in **Figure 19.5** .

Figure 19.5 Stream operations with `filter`

```
IntStream.of -> [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
-> filter -> [3, 1, 1, 5, 9, 5, 3]
-> map -> [9, 1, 1, 25, 81, 25, 9]
-> sum -> 151
```

We now have two different stream modifiers; let's throw in a third. Suppose that we don't want to include any given number more than once in computing this sum. That would normally require some kind of structure like a set to remember values that we have seen so as to eliminate duplicates. But Java provides a specific modifier for this known as `distinct`. The stream that it produces will have values from the original stream in the same order, but with all duplicates removed (only the first occurrence of each value will be included). So we can add this to our code to get:

```
int sum = IntStream.of(3, 1, 4, 1, 5, 9, 2, 6, 5, 3)
    .filter(n -> n % 2 != 0)
    .distinct()
    .map(n -> n * n)
    .sum();
```

Figure 19.6  shows an updated diagram of this new code.

Figure 19.6 Stream operations with `distinct`

```
IntStream.of -> [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
-> filter    -> [3, 1, 1, 5, 9, 5, 3]
-> distinct  -> [3, 1, 5, 9]
-> map       -> [9, 1, 25, 81]
-> sum       -> 116
```

This is a fairly complex operation that we have described without saying much about how it is accomplished. We are requesting the sum of the squares of the odd numbers of this sequence after duplicate numbers have been removed. The stream mechanism and these individual modifiers take care of the rest of the work for us.

As another example, suppose we want to write a method to test whether a number n is prime. By definition a prime is a number that is divisible by 1 and itself but nothing else. Another way of saying that is that primes have exactly two factors. We can express that using streams by producing the values 1 through n , filtering on factors, and seeing whether there are exactly two of them.

```
public static boolean isPrime(int n) {
    return IntStream.range(1, n + 1)
        .filter(x -> n % x == 0)
        .count()
        == 2;
}
```

Remember that the second parameter to `range` is exclusive, which is why we use $n + 1$ to have the stream go all the way up to and including n . It is worth considering the special case of 1. This method says that it is not prime, because there is only one value between 1 and 1 that is divisible by 1. That is the right answer because the value 1 is not considered a prime. This computation is inefficient because it only needs to test values up to the square root of n , but we will delay that discussion for the case study at the end of this chapter.

Using Reduce

Now let's consider the task of computing the factorial of an integer n , which is defined as the product of the integers 1 through n . This is a slightly different task and requires a slightly different approach. In this case, we want to take all of the integers in a stream and combine them into one integer using multiplication. There is a special stream terminator known as `reduce` that combines elements from a stream. Below is a method that computes the factorial using this approach.

```
public static int factorial(int n) {  
    return IntStream.range(2, n + 1)  
        .reduce(1, (a, b) -> a * b);  
}
```

You will notice that the call on `reduce` begins with an extra parameter `1`. This is the starting value to use for the computation. Multiplying by 1 does not change the overall result and it allows the expression to guarantee that it returns a value even if the stream is empty. That means that 0 factorial is correctly reported as 1. It also means that negative factorials are reported as 1 which we could fix with a test that throws an exception in that case because the factorial is undefined for negatives.

There are many details we have left out about streams. For example, we have not yet discussed the problem that some computations do not produce a result. You can generally compute an average or a maximum or a minimum of a sequence of values, but not if the sequence is empty. Java has a notion of an option type that sometimes has a result and sometimes does not, which we will briefly explore in the next section. We will close with a simple variation of our computation on the first ten squares that uses a built-in terminator known as `summaryStatistics` in place of `sum`:

```
System.out.println(IntStream.range(1, 11)
    .map(n -> n * n)
    .summaryStatistics());
```

which produces the following output:

```
IntSummaryStatistics{count=10, sum=385, min=1,  
average=38.500000, max=100}
```

Each of these statistics has a corresponding terminator, such as `average` and `max`. You can read more about them and the other modifiers and terminators in the documentation for the `IntStream` class.

Optional Results

Some of the provided stream terminators have a subtlety that merits investigation. For example, if you want to find and print the largest multiple of 10 in a certain group of integers, you might perform the following stream operations to filter out all the non-multiples of 10 and then print the largest of the remaining numbers:

```
// print largest multiple of 10 in list (does not compile)  
int largest = IntStream.of(55, 20, 19, 31, 40, -2, 62, 30)  
    .filter(n -> n % 10 == 0)  
    .max();  
System.out.println(largest);
```

But you'd find that the preceding code generates a compiler error such as the following:

```
incompatible types: OptionalInt cannot be converted to int
```

This is because the results of some stream terminators like `average`, `findFirst`, `max`, and `min` are undefined if the stream does not contain any data. What if there were no multiples of 10 after doing the filtering? There would be no maximum to return. The terminator could have chosen a default value like 0, but this is not guaranteed to be what the programmer would want.

For this reason, those terminators do not actually return the type of result you would expect, such as an `int` or `double`. The designers of the library needed a way to represent that the result might be undefined. So these terminators actually return values of types called `OptionalInt` and `OptionalDouble`. These “optional” types are small wrapper objects for storing a single `int` or `double` result respectively. Each type has a corresponding method called `getAsInt` and `getAsDouble` to extract the value. Those methods will throw a `NoSuchElementException` if the value is undefined. (There is also a more general optional type called `Optional<T>` that is outside the scope of this chapter.)

The following code properly stores the largest multiple of 10 as an integer:

```
// print largest multiple of 10 in list  
int largest = IntStream.of(55, 20, 19, 31, 40, -2, 62, 30)  
    .filter(n -> n % 10 == 0)  
    .max()  
    .getAsInt();  
System.out.println(largest);
```

A full discussion of these optional types is outside the scope of this chapter, but you can read more about them in the [Java 8 API documentation](#).

19.4 Function Closures

We have seen that Java has extensive rules about the scope of variables that guarantees that every reference to a variable will work out. Consider, for example, the following program that attempts to count how many factors of 10 there are.

```
1  // Attempts to count the factors of 10.
2  // Does not compile; not an example to follow.
3
4  public class BadScope {
5      public static boolean isMultiple(int x) {
6          return n % x == 0;
7      }
8
9      public static void main(String[] args) {
10         int n = 10;
11         int count = 0;
12         for (int i = 1; i <= n; i++) {
13             if (isMultiple(i)) {
14                 count++;
15             }
16         }
17         System.out.println("count = " + count);
```

```
18     }  
19 }
```

This program does not compile. Java complains that the variable `n` referred to in the method `isMultiple` is not defined. You might argue that it is defined in the `main` method and is set up before the function is called. But as we have seen, Java has strict rules about this and requires that the variable `n` be visible in the scope of the method or in the outer class scope. Being available in the scope of a different method isn't good enough.

The usual fix is to pass `n` as a parameter:

```
1  // Counts the factors of 12.  
2  
3  public class GoodScope {  
4      // Returns true if n is a multiple of (is divisible by)  
5      x.  
6      public static boolean isMultiple(int x, int n) {  
7          return n % x == 0;  
8      }  
9  
10     public static void main(String[] args) {  
11         int n = 10;  
12         int count = 0;  
13         for (int i = 1; i <= n; i++) {
```

```

13         if (isMultiple(i, n)) {
14             count++;
15         }
16     }
17     System.out.println("count = " + count);
18 }
19 }

```

This program now compiles and correctly reports that there are four factors of 10 (1, 2, 5, and 10). But consider the following code that uses a stream, a call on `filter`, and a call on the terminator `count` which returns a value of type `long`.

```

int n = 10;
long count = IntStream.range(1, n + 1)
    .filter(x -> n % x == 0)
    .count();
System.out.println("count = " + count);

```

It also correctly reports that there are four factors of 10. There is similar code used in the `isPrime` method included in the previous section. But how does it work? Think about the lambda expression being passed as a parameter to `filter`:

```

x -> n % x == 0

```

This is supposed to represent an independent function. In fact, it looks a lot like the `isMultiple` method from the `BadScope` class. It has a single parameter called `x` and it returns a `boolean` value based on whether the variable `n` is divisible by `x`. It has access to the variable `x` because it is the parameter, but how does it have access to the variable `n`?

You might say that you can see the variable `n` defined in the scope in which this lambda expression occurs and that is the key to understanding this, but it is important to recognize that something special is going on here. Remember that you are passing a function to the `filter` method that it will execute. That `filter` function is in a different class that doesn't have access to the variable `n` that is defined here. So somehow the value of `n` is being passed along with the code.

When you form a lambda expression, some of the variables you refer to will be included as parameters to the function. We call such references *bound variables* because they are each connected to a parameter. Any reference that is not to a parameter in a lambda expression is considered a *free variable*. Variables cannot be completely free of definition because then they would be undefined. But because a lambda expression occurs in the middle of code that potentially has variables defined, the lambda expression can refer to free variables that are defined in its outer scope.

Bound/Free Variable

In a lambda expression, parameters are bound variables while variables in the outer containing scope are free variables.

In our sample lambda expression, the outer code defines a local variable called `n`, which means that the reference to `n` in the lambda expression makes sense. But Java has to perform some work behind the scenes to include this variable definition along with the code. We refer to such a combination as a *closure*.

Function Closure

A block of code defining a function along with the definitions of any free variables that are defined in the containing scope.

Let's use another example that goes to an extreme to make the point. Suppose we have a method called `compute` that takes a function of two arguments as a parameter. We might call it like this:

```
compute((x, y) -> x + y);
```

This lambda expression refers to `x` and `y` which are both parameters, which means they are bound. There are no free variables in this expression. But suppose the code instead had been:

```
int min = 10;
int max = 50;
int multiplier = 3;
compute((x, y) -> Math.max(x, min) * Math.max(y, max) *
multiplier);
```

This lambda expression has references to `x` and `y` just like the other one, but those aren't a problem because they are bound because they are parameters. But it also has references to variables called `min`, `max`, and `multiplier`, which are not parameters of the lambda expression, which means they are free variables. Java has to include those definitions along with the code in order for the compute method to be able to do its job.

For a visual representation, imagine that Java puts together the code of the lambda expression along with the definitions of the parameters and these three free variables into one big object, as shown in

Figure 19.7 .

Figure 19.7 Diagram of a function closure

```
parameters    : (x, y)
free variables: min = 10, max = 50, multiplier = 3
code          : Math.max(x, min) * Math.max(y, max) * multiplier
```

This object is the closure. It contains both the code and the relevant context in which the code occurs. The free variables `min`, `max`, and `multiplier` have been included so that the code can be executed.

We say that they have been *captured* in forming this closure.

The word closure comes from mathematics (e.g., the concept of a transitive closure), but here is a way to think about it that might be helpful to remember the concept. We all seek closure in our lives, which means that we don't want any loose ends from our past left around to upset us. Free variables are loose ends in a lambda expression that would make it impossible to evaluate the code. By including their definition in the closure, we are tying up those loose ends so that there isn't anything left that is undecided. Once we've done that, we've reached a kind of closure with this computation.

We run into potential problems if any of the free variables included in a closure change in value. If the value of a free variable changes, then the computation isn't well defined. For example, it means that the order in which we perform the individual computations for a stream can change the overall result. This relates to the concept of effect-free programming described earlier in this chapter. We want to

write code that does not have these potential sources of inconsistency.

Java requires that any free variable included in a lambda expression has to effectively be a constant. The easiest way to guarantee this is to use values that are defined with the keyword `final`. But the Java compiler will allow you to use ordinary variables as long as your code never assigns those variables a value more than once. This is a helpful restriction, but it still leaves open the possibility that you will have a reference to an object whose state changes. That leads to unpredictable results and should be avoided whenever possible.

19.5 Higher-Order Operations on Collections

You may have noticed that methods like `map`, `filter`, and `reduce` are functions that accept other functions as arguments. As a result, they are known as higher-order functions.

Higher-Order Function

A function that takes another function as an argument.

We have already discussed these higher-order functions previously with examples of streams composed of simple `int` values. In this section, we will explore how similar operations can be performed on arrays and other Java collections. This section also discusses stream terminators that collect stream elements into a collection.

Working with Arrays

Any array can serve as the source for a stream by passing it as a parameter to the method `Arrays.stream`. For example, suppose that you have an array of simple `int` values defined as follows:

```
int[] numbers = {3, -4, 8, 4, -2, 17, 9, -10, 14, 6, -12};
```

Suppose that we want to find the sum of the absolute values of the even integers in this list and we want to exclude any duplicates. That sounds a lot like what we were doing before and we can do it in the same way but using the array as the source of data.

```
int sum = Arrays.stream(numbers)
    .map(n -> Math.abs(n))
    .filter(n -> n % 2 == 0)
    .distinct()
    .sum();
```

We can once again use a diagram to explore what happens on each step of this computation, as shown in [Figure 19.8](#) .

Figure 19.8 Stream operations on an array

```
Arrays.stream -> [3, -4, 8, 4, -2, 17, 9, -10, 14, 6, -12]
-> map        -> [3, 4, 8, 4, 2, 17, 9, 10, 14, 6, 12]
-> filter     -> [4, 8, 4, 2, 10, 14, 6, 12]
-> distinct   -> [4, 8, 2, 10, 14, 6, 12]
-> sum        -> 56
```

Order can matter for these components that modify the stream. For example, if we call `distinct` before we use `map` to convert to the absolute value, then we would miss the fact that there is a -4 and a 4 in the original list. There is a minor efficiency improvement in performing the `filter` operation before the `distinct` operation because `distinct` is easier to perform on a shorter sequence, which is what `filter` provides.

One of these lambda expressions is of particular interest. We call `map` passing it an expression that converts a number to its absolute value:

```
.map(n -> Math.abs(n))
```

Think about this expression. It describes a function that takes a value `n` as a parameter and that returns the value of `Math.abs(n)`. But isn't that exactly what `Math.abs` already does? In other words, `Math.abs` is the function we want to pass to `map`. We don't need to make a new function that is basically a second version of `Math.abs`. Java allows you to avoid this redundancy by directly referring to the `Math.abs` method. We use the name of the class followed by two colons followed by the name of the method. This syntax in Java is called a *method reference*. That means that our code can instead be written as:

```
int sum = Arrays.stream(numbers)
    .map(Math::abs)
    .filter(n -> n % 2 == 0)
    .distinct()
    .sum();
```

Method reference

A syntax to refer to a method so that it can be used as part of a functional expression.

Things work slightly differently for arrays of objects, but we'll explore that in the next section on working with lists. Java cares about the values in a stream but not where those values come from. So a stream whose source is an array of objects would be manipulated in exactly the same way as a stream whose source is a list of objects.

It is also worth noting that the various stream classes have a special terminator called `toArray` that collects the stream contents into an array. For example, the following code:

```
int[] sublist = Arrays.stream(numbers)
    .map(Math::abs)
    .filter(n -> n % 2 == 0)
    .distinct()
    .toArray();
```

will set the variable `sublist` to the following array:

```
[4, 8, 2, 10, 14, 6, 12]
```

Working with Lists

As noted earlier, Java doesn't care what the source of a stream is. It only cares about the kind of data in the stream. But so far we have only looked at streams that contain simple `int` data. Lists store objects, so they tend to be manipulated in a slightly different way.

The `List` interface now includes a method called `stream` that can be used to create a stream of values from a list. For example, suppose that we want to process a list of words. We can use the `Arrays.asList` method to create a list of specific words to manipulate:

```
List<String> words = Arrays.asList("To", "be", "or", "Not",  
"to", "be");
```

Suppose that we simply want to print these words on a line of output. We could use a for-each loop to do so:

```
System.out.print("words:");  
for (String s : words) {  
    System.out.print(" " + s);  
}  
System.out.println();
```

This code produces the following output.

```
words: To be or Not to be
```

Think about what is going on here. We are using a for-each loop to go through every element of the list and to perform some specific task on each of those elements. We can accomplish the same effect by creating a stream from the list and calling the `forEach` method of the stream class to perform the same task expressed using a lambda expression. The stream method `forEach` accepts a lambda that, in turn, accepts a single element from the stream and performs some action on it.

```
System.out.print("words:");  
words.stream()  
    .forEach(s -> System.out.print(" " + s));  
System.out.println();
```

It seems odd to use `forEach` instead of `map`. Isn't this a mapping operation? The answer is yes and this is one of the cases where Java is showing its clunky implementation of functional programming. There are two reasons that we need to use `forEach` instead of `map`. Remember that `map` is a modifier that takes one stream and produces a new stream. We are calling the `System.out.print` method, which has a `void` return type. Because it doesn't return anything, it would not be an appropriate method to use for a `map` operation. More importantly, each stream needs a terminator. In this case, the application of the `print` method is the last thing we want to do with the stream, so we need a terminator rather than a modifier. It's not a bad idea to think of `forEach` as a variation of `map` that serves as a terminator rather than a modifier.

We can make this code a bit more functional by noticing that we are modifying each string to have a space in front of it and then applying the print method to each of those strings. We can describe this more clearly with a call on `map` that modifies each string and then a call on the `System.out.print` method that can be referred to directly using a method reference:

```
System.out.print("words:");  
words.stream()  
    .map(s -> " " + s)  
    .forEach(System.out::print);  
System.out.println();
```

As usual, we can add more modifiers to this stream computation. For example, we can apply the `toLowerCase` method to eliminate any differences in case, the `distinct` method to eliminate duplicates, and the `sorted` method to put the results into sorted order.

```
System.out.print("words:");  
words.stream()  
    .map(String::toLowerCase)  
    .distinct()  
    .sorted()  
    .map(s -> " " + s)  
    .forEach(System.out::print);  
System.out.println();
```

This code produces the following output:

```
words: be not or to
```

There is enough going on here that a diagram is again helpful to understand each individual step. [Figure 19.9](#)  shows the various steps in the computation.

Figure 19.9 Stream operations on a list of strings

```
words.stream -> ["To", "be", "or", "Not", "to", "be"]
-> map        -> ["to", "be", "or", "not", "to", "be"]
-> distinct   -> ["to", "be", "or", "not"]
-> sorted     -> ["be", "not", "or", "to"]
-> map        -> [" be", " not", " or", " to"]
-> forEach    -> prints: " be not or to"
```

Let's explore one last example that shows one of the potential pitfalls that comes up when working with streams. Suppose we want to know the sum of the lengths of the strings in the original list. We can apply the `length` method of the `String` class to each of the strings and then ask for the sum of those values. This seems fairly straightforward:

```
int totalLength = words.stream()
    .map(String::length)
    .sum();
```

Unfortunately, this code does not compile. It says that there is no method called `sum` for the given stream. But doesn't the `length` method of the `String` class return an `int`? So wouldn't the call on

`map` produce a stream of `int` values? If so, it should have the `sum` method we were using in the previous section.

The problem is that Java has different stream types and these types are preserved by modifying components unless you explicitly tell Java to change the stream type. The source is a stream of objects, so the `map` command will produce a stream of objects. In particular, it boxes each of the `int` values returned by the `length` method and gives you a stream of `Integer` objects. If you really want it to produce a stream of simple `int` values, you have to instead call the `mapToInt` method that changes the stream type:

```
int totalLength = words.stream()
    .mapToInt(String::length)
    .sum();
```

This version of the code works properly, setting the variable `totalLength` to `13`. Making a diagram is again helpful to consider exactly what is happening here, as shown in [Figure 19.10](#) .

Figure 19.10 Stream operations on a list of strings mapped to integers

```
words.stream -> ["To", "be", "or", "Not", "to", "be"]
  -> mapToInt -> [2, 2, 2, 3, 2, 2]
  -> sum       -> 13
```

Notice that the original stream contains strings, but after the call on `mapToInt`, we have a stream of `int` values. That stream can be terminated with a call on `sum`, as we did in the examples in the last section.

As noted earlier, this short chapter is not meant to cover all of the details of these Java 8 constructs. We are exploring the important concepts with a representative subset of these operations. But if you keep in mind the idea of streams changing their types, then you will probably understand why there are methods with names like `mapToInt`, `mapToDouble`, `mapToLong`, and `mapToObj` and why it is sometimes handy to call the modifier called `boxed` that converts a stream of primitive values into its corresponding wrapped objects.

We saw that you can collect the values in a stream into an array by using the terminator called `toArray`. There is a similar method called `collect` that allows you to collect the results in a collection. You need to construct an appropriate `Collectors` object by calling an appropriate method such as `toList` or `toSet`. For example, using our words example from before, we can convert all of the words to lowercase and collect the results in a set.

```
Set<String> words2 = words.stream()
    .map(String::toLowerCase)
    .collect(Collectors.toSet());
System.out.println("word set = " + words2);
```

This code produces the following output:

```
word set = [not, be, or, to]
```

If you want to specify exactly which type of collection to produce, you can do so with a call that passes in a reference to the constructor of the appropriate class. For example, if you want to guarantee that you get a `TreeSet` that keeps the keys in sorted order, you would instead say:

```
Set<String> words2 = words.stream()  
    .map(String::toLowerCase)  
    .collect(Collectors.toCollection(TreeSet::new));  
System.out.println("word set = " + words2);
```

In which case the output becomes:

```
word set = [be, not, or, to]
```

As with the other examples, we are only scratching the surface here of the details of how all of this works, but these examples serve as good indicators of the kind of operations that can be performed using streams.

As you can see from the examples in this section, functional style programming is particularly helpful for processing of collections like arrays and lists. The trio of higher-order functions `map`, `filter`, and `reduce` provide a particularly powerful set of tools for solving a wide variety of computational tasks on collections.

Working with Files

Java 8 provides a new facility for reading the lines of a file into a stream of strings. Importing the package `java.nio.file` will give you access to the `Files` class, which contains a static method `lines` that returns a stream of the lines of a file. The syntax is the following:

```
Files.lines(Paths.get("<filename>"))
```

The `Files.lines` method throws an `IOException` if the file does not exist or cannot be read, so you must surround it with a `try/catch` block or use a throws clause in your method heading. For example, suppose we have a file in the current directory named `haiku.txt` that stores the following three lines of text:

```
haiku are funny  
but sometimes they don't make sense  
refrigerator
```

The `Files.lines` method would return the file's three lines as a stream containing three string elements. The following code would read and print every line of `haiku.txt` using a stream:

```
// print every line of the file
try {
    Files.lines(Paths.get("haiku.txt"))
        .forEach(System.out::println);
} catch (IOException ioe) {
    System.out.println("Could not read file: " + ioe);
}
```

If you want to process each line, the standard stream operations like `map`, `filter`, and `reduce` can be used on the strings in the stream. For example, the following code finds out the length of longest line in the file:

```
// find longest line in the file
int longest = Files.lines(Paths.get("haiku.txt"))
    .mapToInt(String::length)
    .max()
    .getAsInt();
```

Figure 19.11  summarizes the stream operations and their results. Recall that we need to use `getAsInt` at the end of our stream calls because the `max` terminator returns an `OptionalInt` to represent the possibility that there might be no lines in the file.

Figure 19.11 Stream operations on the lines of a file

```
Files.lines -> ["haiku are funny",  
               "but sometimes they don't make sense",  
               "refrigerator"]  
-> mapToInt -> [15, 35, 12]  
-> max       -> 35
```

19.6 Case Study: Perfect Numbers

To complete our exploration of functional programming, we will look at a classic problem from mathematics that is known to take a lot of computational power to solve. We are going to write a program that looks for perfect numbers. A perfect number is defined as one that is equal to the sum of its divisors other than itself. For example, the divisors of 6 are [1, 2, 3, 6]. If you exclude 6, the other divisors add up to $6(1 + 2 + 3)$. In fact, 6 is the smallest perfect number. The next perfect number is 28 whose divisors are [1, 2, 4, 7, 14, 28].

If you search for “perfect number” in your web browser, you will find several web sites that chronicle the fascination that many people have had over the years with this concept. Some ancient Greeks believed that the world was created in 6 days because 6 is the first perfect number. St. Augustine repeats this claim in his writings. The ancient Greeks also believed that the moon completes an orbit every 28 days because 28 is the second perfect number.

The first four perfect numbers have been known since ancient times, but the earliest known references to the fifth perfect number date from the fifteenth century. In this section we will write a program to find the fifth perfect number. In doing so, we will be able to explore one of the biggest benefits of functional programming: that it can

speed up the execution of programs that require a lot of computational power.

Computing Sums

This problem, as with many problems from mathematics, lends itself naturally to a functional approach. In this first version, let's write code that finds the sum of the divisors of a number not including the number itself and include debugging code that will allow us to have confidence that it is working correctly.

We can use `Intstream.range` to produce a stream of integers and filter it with a test for divisibility and then compute the sum. Let's put it into a method so that we can call it easily:

```
public static int sumDivisors(int n) {  
    return IntStream.range(1, n)  
        .filter(x -> n % x == 0)  
        .sum();  
}
```

Normally we would have range go up to `n+1` so that it will include `n`, but in this case we want to exclude `n` from the sum because we are looking for the sum of the divisors other than the number itself.

In the `main` method, we can compute the sums for the first 10 integers and print them out again using a call on `IntStream.range` and a call on `map` to apply our function:

```
IntStream.range(1, 11)
    .map(Perfect1::sumDivisors)
    .forEach(n -> System.out::println.print(n + " "));
System.out.println();
```

Notice that we terminate this stream with a call on `forEach` that prints each number with a space after it. This will produce an extraneous space at the end of the line, but that's not a problem for us at this stage in development. Also notice that the call on `map` takes a method reference to the function to be applied. We use the name of the class (the one we are writing) along with the double colon operator and the name of the method from this class that we want to call.

The code above produces the following output:

```
0 1 1 3 1 6 1 7 4 8
```

We could hand-check that these values are correct, but it is helpful to build some debugging code into our solution that will allow us to see more clearly exactly what is going on in this computation. Java

provides a special method called `peek` that is particularly helpful for this kind of debugging. In some sense, the `peek` method has no effect on a stream at all. It produces as output the same stream that it is given as input. But it allows you to pass it a function to be executed on each value in the stream. In our case, we would like to do some printing of individual stream values. We see 10 numbers above and can figure out which one goes with each input value to the function, but we can use `peek` to confirm this more directly.

The code below includes a call on `peek` that prints the number and an equals sign before it maps that number to its divisor sum and prints it.

```
IntStream.range(1, 11)
    .peek(n -> System.out.print(n + "="))
    .map(Perfect1::sumDivisors)
    .forEach(n -> System.out.print(n + " "));
System.out.println();
```

It produces the following output.

```
1=0 2=1 3=1 4=3 5=1 6=6 7=1 8=7 9=4 10=8
```

This output indicates that the sum of divisors for 1 is 0, the sum of divisors for 2 is 1, the sum of divisors for 3 is 1, and so on. This

sounds wrong, but remember that we are excluding the number itself from these divisors. We can make this even clearer by adding a call on `peek` to our stream that is computing divisors:

```
return IntStream.range(1, n)
    .filter(x -> n % x == 0)
    .peek(x -> System.out.print(x + ","))
    .sum();
}
```

This version of the code produces the following output:

```
1=0 2=1,1 3=1,1 4=1,2,3 5=1,1 6=1,2,3,6 7=1,1 8=1,2,4,7 9=1,3,4
10=1,2,5,8
```

This output is a little hard to read as well, but keep in mind what we have added. As the method that sums up the divisors encounters a new divisor, it prints it with a comma afterward. For example, when it is finding the divisors of 10 it finds the divisors 1, 2, and 5. So it produces the output “1,2,5,” which includes an extra comma at the end. Then the code we have in `main` prints the actual result, which is 8. We can see that $1 + 2 + 5$ is equal to 8, so we know that this output verifies that the code is working properly.

It is useful to look at a diagram for the specific computation of the divisors of 10. **Figure 19.12**  shows the steps of the stream

computation.

Figure 19.12 Stream operations to compute sum of divisors of 10

```
IntStream.range -> [1, 2, 3, 4, 5, 6, 7, 8, 9]
    -> filter -> [1, 2, 5]
    -> peek    -> [1, 2, 5] with side effect of printing "1,2,5,"
    -> sum     -> 8
```

As the diagram above indicates, the call on `peek` is producing a side effect. Normally we want to write code that is effect-free without such side effects, but in this case it is a useful way to make the computation more visible for debugging purposes.

Consider a similar diagram for the overall computation being performed by the `main` method. **Figure 19.13**  shows the stream operations.

Figure 19.13 Stream operations to compute sum of divisors of 10

```
IntStream.range -> [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    -> peek      -> [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
                    with side effect of printing "1=2=3=4=5=6=7=8=9=10="
    -> map       -> [0, 1, 1, 3, 1, 6, 1, 7, 4, 8]
    -> forEach  -> prints to System.out: "0 1 1 3 1 6 1 7 4 8"
```

Figure 19.13  is confusing. It implies that it prints all of the text from the call on `peek` before it calls `map` and only later prints all of the answers through the call on `forEach`. A diagram can only convey so much of a complex process like the processing of a stream. The diagram is showing the resulting stream on each step, but that is different from the order of operations. This can get very tricky to understand, but here is a brief explanation. Nothing happens until Java encounters the terminator. When it gets to the call on `forEach`, it works backward to get a value from the stream to process. It goes all the way back to the source which was the call on `range`. The `range` method sends out the value 1, which is given to `peek` to print “1=”, and then it passes to `map` which converts it to 0, and then it makes it back to `forEach` that prints the 0. At this point in time we have as output “1=0”. Then the `forEach` method asks for another value and we begin the process over again using the value 2, then 3, then 4, all the way up to 10. So the right way of thinking of this is that individual values of the stream pass through each of the different modifiers in sequence before the next value from the stream is processed.

There is a great moral of the story here. The main reason this is confusing is that the output generated by `peek` is mixed in with the output generated by the terminator in a surprising way. And that output generated by `peek` is a side effect. If we restrict ourselves to effect-free code that doesn't have side effects, then we rarely need to think about these issues of ordering.

Below is the complete code for this version that prints the sums for the numbers 1 through 10 including debugging information.

```
1 // Prints the sum of factors of the integers 1 - 10.
2 // Initial version.
3
4 import java.util.stream.*;
5
6 public class Perfect1 {
7     public static void main(String[] args) {
8         IntStream.range(1, 11)
9             .peek(n -> System.out.print(n + "="))
10            .map(Perfect1::sumDivisors)
11            .forEach(n -> System.out.print(n + " "));
12        System.out.println();
13    }
14
15    // returns the sum of the proper divisors of n
16    public static int sumDivisors(int n) {
17        return IntStream.range(1, n)
18            .filter(x -> n % x == 0)
19            .peek(x -> System.out.print(x + ", "))
20            .sum();
21    }
22 }
```

Incorporating Square Root

It seems like we are almost done. We can modify `main` so that it looks at a lot of integers (say, up to 1 million) and filters on those that are equal to their sum of divisors. We can also use the `System.currentTimeMillis` method introduced in [Chapter 13](#)  to keep track of how much time we spend computing the result. We can temporarily turn off our debugging code by commenting it out and include the following code in `main`.

```
long start = System.currentTimeMillis();
IntStream.range(1, 1000001)
    .filter(n -> n == sumDivisors(n))
    .forEach(System.out::println);
double elapsed = (System.currentTimeMillis() - start) / 1000.0;
System.out.println();
System.out.println("time = " + elapsed);
```

When one of the authors ran this on his laptop computer, it reported taking 2260.112 seconds to complete, which is over 37 minutes. It correctly reported the first four perfect numbers, but it didn't manage to find the fifth. And it isn't going to find the fifth in a reasonable amount of time.

We need to improve the efficiency of our computation. The easiest way to do this is to notice that each divisor that is less than the

square root of a number is paired with a divisor that is greater than the square root. If, for example, we are looking for the divisors of 100, we find that the divisor 1 is paired with 100, the divisor 2 is paired with 50, the divisor 4 is paired with 25, and so on. So instead of finding each divisor by checking all the way up to the number, we can instead find pairs of divisors by checking up to the square root. If you search for the divisors in this way, for each divisor x you find, you know that you have also found its pair, whose value is n / x ; so our modified code contains an additional call to `map` that adds in each divisor's pair to our sum.

Remember that we want to exclude the number n itself from the sum. It is always paired with 1, so an easy way to do this is to start our search for divisors starting at 2 rather than 1. If we do that, however, then we have to remember to add 1 to the final result because 1 is supposed to be included in the sum. The code below implements this strategy and updates the call on `peek` to report the pairs of divisors.

```
// returns the sum of the proper divisors of n
// (optimized version that computes up to square root only)
public static int sumDivisors(int n) {
    int root = (int) Math.sqrt(n);
    int sum = IntStream.range(2, root + 1)
        .filter(x -> n % x == 0)
        .peek(x -> System.out.print(x + "-" + n / x + ", "))
        .map(x -> x + n / x)
```

```
.sum();  
return sum + 1;  
}
```

We can go back to our old client code that shows output for the numbers 1 through 10, which produces the following output.

```
1=1 2=1 3=1 4=2-2,5 5=1 6=2-3,6 7=1 8=2-4,7 9=3-3,7 10=2-5,8
```

This output is different from before in that it doesn't include 1 at the beginning of each list of divisors. That's because we are handling 1 as a special case, as described previously. But there is something else to notice. It is getting the wrong answers for some of these values. In particular, it has the wrong answer for 1, 4, and 9. You might be able to notice what those three numbers have in common, but you can also figure this out by looking at one of them in detail. Let's take 4. Its divisors other than itself are 1 and 2, so that should add up to 3. Instead we are getting an indication that it has a pair of divisors 2 and 2 and that it adds up to 5 when you include the special case of 1.

The problem comes for numbers that are perfect squares. It is true that every divisor that is strictly less than the square root of a number has a different divisor that it is paired with that is strictly larger than the square root. But for numbers that are perfect squares, their square root isn't paired with another divisor. This is a special case

that we can handle after the main computation of `sum` by adding the following code.

```
if (n == root * root) {  
    sum = sum - root;  
}
```

Below is the complete program that checks numbers up to 1 million. This new version takes only 4.493 seconds to run, which is quite a bit better than 37 minutes.

```
1 // Prints the sum of factors of the integers 1 - 1,000,000.  
2 // Second version with optimizations to compute the sum of  
3 // n's factors more quickly by examining only up to  
sqrt(n).  
4 import java.util.stream.*;  
5  
6 public class Perfect2 {  
7     public static void main(String[] args) {  
8         long start = System.currentTimeMillis();  
9         IntStream.range(1, 1000001)  
10             .filter(n -> n == sumDivisors(n))  
11             .forEach(System.out::println);  
12         double elapsed = (System.currentTimeMillis() -  
start)  
13             / 1000.0;
```

```
14      System.out.println();
15      System.out.println("time = " + elapsed);
16  }
17
18      // returns the sum of the proper divisors of n
19      public static int sumDivisors(int n) {
20          int root = (int) Math.sqrt(n);
21          int sum = IntStream.range(2, root + 1)
22              .filter(x -> n % x == 0)
23              .map(x -> x + n / x)
24              .sum();
25          if (n == root * root) {
26              sum = sum - root;
27          }
28          return sum + 1;
29      }
30  }
```

Just Five and Leveraging Concurrency

We are basically ready to see if we can actually find the fifth perfect number. We have checked up to 1 million and didn't find it. We could try going up to 10 million or 100 million or some other large number. But Java provides a nice alternative.

We have been using a call on `IntStream.range` to specify a specific range of numbers to use. We can instead use the method `IntStream.iterate` to produce a stream containing all positive integers. The `iterate` method takes a starting value and an update function. So the source of our stream will be the following call.

```
IntStream.iterate(1, n -> n + 1)
```

This is potentially an infinite stream of numbers because we say to start at 1 and to update by replacing n with $n + 1$, but we never say how it stops. When you work with a stream like this, you have to include a special modifier that limits how many values it produces. Not surprisingly, this modifier is called `limit`. The following code prints the first five integers that are equal to the sum of their divisors (in other words, the first five perfect numbers):

```
IntStream.iterate(1, n -> n + 1)
    .filter(n -> n == sumDivisors(n))
    .limit(5)
    .forEach(System.out::println);
```

This is another place where it is important to understand in general the order of operations. If you think of the call on `iterate` as producing all of its result before you ever get to the call on `filter`, then this code makes no sense. You have to understand this code in

terms of individual values being generated by `iterate` and then passed to `filter`. When one of those values passes the test, it is passed on to `limit` and then to `forEach` which prints it. After the `limit` method has produced five values, it stops requesting new values to be checked from the `iterate` method.

We can throw in a call on the modifier `unordered` because it makes the `limit` method easier to optimize so that it potentially runs a bit faster. Combining all of this and running the program with the timing code, it ends up producing output like the following. The program finds the fifth perfect number and takes less than 11 minutes on one of the authors' laptops.

```
6
28
496
8128
33550336
time = 629.301
```

But we can do even better. Computer scientists have realized for a while now that speeding up a single processor cannot be achieved indefinitely and that eventually we will have to leverage the power of concurrency. It is typical now for laptops to be dual-core or quad-

core or some other form of multi-core where there is more than one processor. But it isn't easy to take advantage of multiple processors.

The pitfalls of concurrency are beyond the scope of this discussion, but think about something as simple as preparing a meal to be served at a restaurant. It is easy to see how one chef would complete the task by doing everything. Adding additional chefs might be helpful if you can divide up the task somehow. For example, one might prepare an appetizer while another prepares the main dish and a third prepares a dessert. Even in this case the timing matters because if the dessert is completed quickly and the main dish comes out last, then it is possible that the dessert will spoil because the customer won't want to eat it until the end of the meal. And if you try to have a dozen cooks work on the meal you generally end up with the well-known disaster of "too many cooks in one kitchen."

When Google was faced with the problem of performing massive calculations over vast databases, they quickly turned to concurrency with multiple machines to speed things up. They realized that many computations can be decomposed into a mapping operation and a reducing operation similar to the approach we have described in this chapter. For example, if you want to count how many web pages have a particular search phrase, you can map a function over the pages that returns 0 or 1 depending on whether the search term appears in that particular page and then you can reduce all of those 0s and 1s using simple addition. They built a system that they call MapReduce that uses exactly this approach. Typical computations are executed with hundreds of processors working on each problem.

An open source version of their system called Hadoop has also been popular for applying concurrency effectively to large-scale computations.

The increasing importance of concurrency has led many to realize that functional programming is a powerful way to take advantage of parallel computation. Its emphasis on effect-free programming allows it to avoid many of the common pitfalls of running code on multiple processors. The model of expressing a computation in terms of mapping, filtering, and reducing lends itself naturally to a concurrent solution because usually these operations can be performed in parallel without affecting the overall result.

Many people enjoy functional programming because they like to express problems in a functional manner. Others appreciate the fact that you can write short code that has a certain elegance to it because complex computations can be expressed very concisely. But in the world of modern computing, the truly compelling reason to study functional programming is that it provides a practical solution to the problem of taking advantage of concurrency. In other words, concurrency is the “killer app” for functional programming that convinced the IEEE/ACM joint task force among others that every undergraduate majoring in computer science needs to understand the basics of functional programming.

It is therefore fitting to end this chapter by noting that the final version of the program has one minor change. We insert a modifier

called `parallel` in the block in `main` that is searching for perfect numbers:

```
IntStream.iterate(1, n -> n + 1)
    .parallel()
    .unordered()
    .filter(n -> n == sumDivisors(n))
    .limit(5)
    .forEach(System.out::println);
```

This extra call tells Java that it would be okay to perform this computation in parallel. It's okay for Java to use as many cooks as it has available to work on the individual problems. Imagine, for example, that a thousand different processors are working on this. There is no reason you can't say, "Processor #1, you figure out whether 1 is a perfect number while processor #2 figures out whether 2 is and processor #3 figures out whether 3 is, etc., etc."

Notice, again, that this style of programming is characterized by the lack of specificity of how it is to be accomplished. We describe the source of the numbers to examine, the test to perform, the fact that we want five answers, the fact that we want to print each one, and the fact that the computations can be performed in parallel. But we leave it up to the computer to figure out how best to optimize the actual computations. Many programmers resist this style because many of us are control freaks who want to say exactly how everything is done. It turns out that loosening the reins and allowing

the computer to optimize it for us often leads to much better results if we have expressed what we want in a way that preserves flexibility for how to get there. That is one of the things that functional programming is best suited for.

When this final version was run on the same laptop, it produced the following output on one of the authors' computers.

```
6
28
496
8128
33550336
|
time = 170.636
```

This is 3.7 times faster than it was before. Not surprisingly, the author's laptop is a quad-core machine with four different processors. The final program appears below.

```
1 // This program searches for the first five perfect numbers.
It uses
2 // a functional programming approach including a
specification that
3 // the computation can be performed in parallel to speed it
up.
```

```
4
5 import java.util.stream.*;
6
7 public class Perfect {
8     public static void main(String[] args) {
9         long start = System.currentTimeMillis();
10        IntStream.iterate(1, n -> n + 1)
11            .parallel()
12            .unordered()
13            .filter(n -> n == sumDivisors(n))
14            .limit(5)
15            .forEach(System.out::println);
16        double elapsed = (System.currentTimeMillis() -
17start)
18            / 1000.0;
19        System.out.println();
20        System.out.println("time = " + elapsed);
21    }
22    // pre: n >= 1
23    // post: returns the sum of the proper divisors of n
24    //        (i.e., not
25    //        including n itself)
26    public static int sumDivisors(int n) {
27        int root = (int) Math.sqrt(n);
28        int sum = IntStream.range(2, root + 1)
29            .filter(x -> n % x == 0)
30            .map(x -> x + n / x)
```

```
30         .sum();
31         if (n == root * root) {
32             sum = sum - root;
33         }
34         return sum + 1;
35     }
36 }
```

Chapter Summary

Functional programming is a style that emphasizes the use of functions or methods to decompose problems. Java 8 added new constructs to the language to support functional programming.

A side effect is a change made to the state of a program that occurs when a function or method is called, such as modifying a global variable or printing output. Functional programmers try to avoid side effects as much as possible.

A first-class function is one that can be treated like other types of data, such as being passed as a parameter or composed with other functions.

Java provides a shorthand syntax for defining anonymous functions called lambda expressions or lambdas for short.

A stream is a sequence of elements from a data source that supports aggregate operations. An array, collection, string, range of integers, or many other sources of data can be converted into a stream.

Typical operations performed on a stream include map (apply an operation to each element), filter (keep or remove some elements based on various criteria), and reduce (combine multiple elements into a single element).

A closure is a function definition along with the definitions of any variables declared outside the function (“free variables”) that the function utilizes.

A higher-order function is one that accepts another function as an argument. Java supports a limited form of higher-order functions through a feature called method references.

Self-Check Problems

19.1 Effect-Free Programming

1. Why do functional programmers want to avoid side effects?
2. Why is calling `System.out.println` considered a side effect?
Does this imply that calling `System.out.println` is a bad thing?
3. What side effect does the following function have? How could it be rewritten to avoid side effects?

```
// Doubles the values of all elements in an array.  
public static void doubleAll(int[] a) {  
    for (int i = 0; i < a.length; i++) {  
        a[i] = 2 * a[i];  
    }  
}
```

4. Rewrite the `SideEffect` program from this section so that it does not contain any side effects. Rather than modifying a global variable, make the function accept the value of `x` to use as a parameter.

19.2 First-Class Functions

5. What change must be made to the math drilling program from this section for it to support subtraction problems?
6. Write a lambda expression that converts an integer into the square of that integer; for example, 4 would become 16.
7. Write a lambda expression that accepts two integers and chooses the larger of the two; for example, if given 4 and 11, it would return the 11.
8. Write a lambda expression that accepts two strings representing a first and last name and concatenates them together into a string in “Last, First” format. For example, if passed `"Cynthia"` and `"Lee"`, it would return `"Lee, Cynthia"`.

19.3 Streams

9. Is a stream the same as an array? How are they similar, and how are they different?
10. What value is stored into the variable `result` by the following code?

```
int result = IntStream.of(1, 2, 3, 4, 5, 6, 7)
    .map(n -> n / 2)
    .distinct()
    .count();
```

11. Write a piece of code that uses stream operations to compute the sum the negations of a stream of integers. For example, if the stream contains `{2, 4, -1, 8}`, the sum to compute is $-2 + -4 + 1 + -8 = -13$.
12. Write a piece of code that uses stream operations to count the number of even integers in a stream. For example, if the stream contains `{18, 1, 6, 8, 9, 2}`, there are 4 even integers.
13. What value is stored into the variable `result` by the following code?

```
int result = IntStream.of(3, -4, 8, -6, 1)
    .map(n -> Math.abs(n))
    .reduce(0, (a, b) -> a + 2 * b);
```

14. The following code does not compile. Why not? How must it be modified?

```
double avg = DoubleStream.of(3.1, -4.5, 8.9, -6.2, 1.0)
    .map(n -> Math.abs(n))
    .average();
```

19.4 Function Closures

15. What is the difference between a free variable and a bound variable?

16. What are the free variables and bound variables in the lambda function in the following code?

```
int a = 1;
int b = 2;
compute((c, d) -> c + b - a);
```

17. The following code does not compile. Why not? What is upsetting the compiler?

```
int a = 10;
int b = 20;
int sum = IntStream.of(1, 2, 3, 4, 5)
    .map(n -> n + b - (++a))
    .sum();
```

19.5 Higher-Order Operations on Collections

18. What is the output of the following code?

```
int[] a = {10, -28, 33, 28, -49, 56, 49};
Arrays.stream(a)
    .map(Math::abs)
    .forEach(System.out::println);
```

-
19. Modify the code from the previous problem so that it does not print any duplicate values.
20. Suppose you have an array of integers called `numbers`. Write a piece of code that uses stream operations to make a new array called `positives` that stores only the positive integers from `numbers`.
21. Suppose you have a list of strings declared as follows. Write code to use stream operations to print all of the four-letter words in the list.

```
List<String> list = Arrays.asList("four", "score", ..., "ago");
```

22. Write code to print all lines from the file `notes.txt` that are at least 40 characters long.
23. The following code contains four problems causing it to fail to compile. What are they?

```
public static int longestLineLength(String filename) {  
    return Files.lines(filename)  
        .map(String::length)  
        .max();  
}
```

Exercises

1. Write a method `printDoubled` that uses stream operations to print twice the value of each element of array of integers. For example, if the array passed is `{2, -1, 4, 16}`, print `4 -2 8 32`.
2. Write a method `sumAbsVals` that uses stream operations to compute the sum of the absolute values of an array of integers. For example, the sum of `{-1, 2, -4, 6, -9}` is `22`.
3. Write a method `largestEven` that uses stream operations to find and return the largest even number from an array of integers. For example, if the array is `{5, -1, 12, 10, 2, 8}`, your method should return `12`. You may assume that the array contains at least one even integer.
4. Write a method `totalCircleArea` that uses stream operations to compute and return the sum of the areas of a group of circles, rounded to the nearest whole number. Your function accepts an array of real numbers representing the radii of the circles. For example, if the array is `{3.0, 1.0, 7.2, 5.5}`, return `289.0`. Recall that the area of a circle of radius r is πr^2 .
5. Write a method `countNegatives` that uses stream operations to count how many numbers in a given array of integers are

negative. For example, if the array is `{5, -1, -3, 20, 47, -10, -8, -4, 0, -6, -6}`, return `7`.

6. Write a method `pigLatin` that uses stream operations to convert a `String` parameter into its “Pig Latin” form. For this problem we’ll use a simple definition of Pig Latin where the first letter should be moved to the end of the word and followed by “ay.” For example, if the string passed is `"go seattle mariners"`, return `"o-gay eattle-say ariners-may"`.
7. Write a method `countVowels` that uses stream operations to count the number of vowels in a given string. A vowel is an A, E, I, O, or U, case-insensitive. For example, if the string is `"SOO beautiful"`, there are seven vowels.
8. Write a method `toSortedForm` that uses stream operations to convert a `String` parameter into a sorted form with its letters in alphabetical order. For example, if the string passed is `"tennessee"`, return `"eeeennsst"`.
9. Write a method `stdev` that computes the standard deviation of an array of real numbers. The formula for computing a standard deviation σ of N values is the following, where x_i represents each i^{th} element and μ represents the arithmetic mean (average) of all the elements:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

10. Write a method `glueReverse` that accepts a `List` of strings as its parameter and uses stream operations to return a single string consisting of the list's elements concatenated together in reverse order. For example, if the list stores `["the", "quick", "brown", "fox"]`, you should return `"foxbrownquickthe"`.
11. Write a method `theLines` that accepts a file name as a parameter and uses stream operations to return a count of the number of lines in the file that start with the word "The", case-insensitive.
12. Write a method `fourLetterWords` that accepts a file name as a parameter and returns a count of the number of unique lines in the file that are exactly four letters long. Assume that each line in the file contains a single word.
13. Write a method `firstFive` that accepts a file name string as its parameter and uses stream operations to print the first five non-blank lines of the file. Do not use any loops or collections; perform the work entirely using streams.

Programming Projects

1. Write a file searching program that uses streams to efficiently search a set of files for a given substring. Write two versions of the code, one that sequentially reads each file with a `Scanner` and checks each line to see if it contains the substring, and a second that uses streams to open all of the files and search the lines using stream operations. The most efficient version of the stream code will open all of the files in parallel. Test how much more efficient the streams are than the `Scanner` by using `System.currentTimeMillis` to measure the elapsed time for both versions of the code when run on a collection of large files, printing output such as the following:

```
Searching 15 files for "the" using Scanner:
there were 84530 total matching lines.
Took 546 ms.

Searching 15 files for "the" using streams:
there were 84530 total matching lines.
Took 160 ms.
```

2. Write a program that prompts the user for an integer value n and that reports the sum of the first n prime numbers, reporting a sum of 0 if the user enters a value less than 1.

Structure your program to be similar to the case study, using an iterating function to produce the sequence 1, 2, 3, . . . , n and filtering using the `isPrime` method that appears at the end of [section 19.3](#). Don't include the `unordered` modifier because we want the first n primes, not some other combination of n primes. Include timing code and print how long it takes to compute the sum.

Once you have a working program, explore the following efficiency improvements and note how the time changes with each using a fairly large value of n such as 10,000:

- Modify `isPrime` to check only up to the square root, as in the case study (remember that 1 is not a prime).
- Modify the iterating function to examine only odd numbers and manually add 2 to the sum (because 2 is the only even prime).
- Modify the overall iteration to include the `parallel` modifier, as in the case study. Note that the benefits of parallel execution might not be evident for this computation because the problem size is fairly constrained (values of n greater than 20,043 will lead to integer overflow).